# Capgemini AI-Assisted Coding Master Guide

Campus Recruitment Drive 2026–2027 | Trainer Reference & Student Question Bank

**Target: 25 Core Patterns**
**Batch: 2027 Candidates**

**Training Methodology & Evaluation Metrics:** Capgemini's new AI-assisted coding format tests 4 critical pillars: (1) **Framing & Prompt Quality** (clear specification of boundaries and constraints), (2) **Algorithm Selection** (guiding time/space complexity), (3) **Code Review & Rectification** (spotting AI hallucinations and syntax bugs), and (4) **Edge Verification** (empty strings, negative values, large bounds). Instruct students to formulate prompts using the structured formula: *[Goal] + [Language] + [Algorithmic Strategy / Complexity] + [Constraints & Edge Cases]*.

## Module 1: Frequency, Strings & Character Transformations

### Q1. Run-Length String Compression (e.g., "aabbbbeeeeff" → "a2b4e4f2")

STRING PROCESSING

**Optimal Student Prompt:** "Write a Java solution to compress a string of repeated characters by printing the character followed by consecutive count (e.g. 'aabbbbeeeeff' -> 'a2b4e4f2'). Single occurrences omit '1' or print '1' based on exact match. Handle single pass in O(n) time and O(1) auxiliary space without regex."

```
public static String compress(String s) { if (s == null || s.isEmpty()) return ""; StringBuilder sb = new StringBuilder(); int count = 1; for (int i = 0; i < s.length(); i++) { if (i + 1 < s.length() && s.charAt(i) == s.charAt(i + 1)) count++; else { sb.append(s.charAt(i)).append(count); count = 1; } } return sb.toString(); }
```

**Review Check:** Watch for boundary exceptions at `s.charAt(i+1)` and ensure single-char inputs (e.g. "a") don't throw an OutOfBounds index.

### Q2. Move All Hashes ('#') to Front of String (Capgemini Classic)

TWO-POINTER / ARRAY

**Optimal Student Prompt:** "Write a C++ function `void moveHashes(string &str)` that moves all '#' characters to the front while preserving the original relative order of all alphabetic characters. Solve in-place or O(n) single pass with O(1) extra space."

```
void moveHashes(string &str) { string hashes = "", chars = ""; for (char c : str) { if (c == '#') hashes += '#'; else chars += c; } str = hashes + chars; // In-place: scan backwards & fill to achieve strict O(1) space }
```

**Review Check:** Verify relative order of remaining letters is strictly preserved and string contains no missing characters.

### Q3. First Element Whose Second Occurrence Appears First

HASHING / SETS

**Optimal Student Prompt:** "Write a Java function to find the first recurring character/integer in an array where the duplicate is encountered earliest. Use a HashSet for O(n) average time and O(n) space. Return -1 if all elements are distinct."

```
public static int firstRepeatedEarly(int[] arr) { HashSet<Integer> seen = new HashSet<>(); for (int num : arr) { if (seen.contains(num)) return num; // First whose 2nd occurrence occurs seen.add(num); } return -1; }
```

**Review Check:** Make sure candidate does not confuse with 'first duplicate by first index appearance' (which requires nested loop or 2 passes).

## Module 1 Continued: String Manipulations & Patterns

### Q4. Anagram Checking Ignoring Case & Special Symbols

HASH MAP / FREQUENCY

> **Optimal Student Prompt:** "Write a Java program to verify if two strings are valid anagrams. Ignore case sensitivity and non-alphanumeric characters. Implement using a fixed 26-element integer frequency array in O(n) time and O(1) extra memory."

```
public static boolean isAnagram(String s1, String s2) { int[] freq = new int[26]; int count = 0; for (char c : s1.toLowerCase().toCharArray()) if (c >= 'a' && c <= 'z') { freq[c - 'a']++; count++; } for (char c : s2.toLowerCase().toCharArray()) { if (c >= 'a' && c <= 'z') { freq[c - 'a']--; count--; } } if (count != 0) return false; for (int val : freq) if (val != 0) return false; return true; }
```

**Review Check:** Prompt must specify handling of punctuation; AI frequently attempts expensive `replaceAll("[^a-zA-Z]", "")` regex.

### Q5. Reverse Words in a Given Sentence Preserving Spaces

TWO POINTERS

> **Optimal Student Prompt:** "Write a C++ function `string reverseWords(string s)` that reverses the order of words in a sentence while normalizing multiple consecutive spaces to a single space, stripping leading/trailing whitespace. Target O(n) time."

```
string reverseWords(string s) { stringstream ss(s); string word, res = ""; while (ss >> word) { res = word + (res.empty() ? "" : " ") + res; } return res; }
```

**Review Check:** Confirm string stream handles trailing/leading white space cleanly without appending null strings.

### Q6. Longest Palindromic Substring Extraction

DYNAMIC PROGRAMMING

> **Optimal Student Prompt:** "Provide a Java method to find the longest palindromic substring in string S. Use the Expand Around Center technique to achieve O(n^2) time and O(1) auxiliary space instead of O(n^2) DP table memory."

```
public static String longestPalindrome(String s) { if (s == null || s.length() < 1) return ""; int start = 0, end = 0; for (int i = 0; i < s.length(); i++) { int len1 = expand(s, i, i), len2 = expand(s, i, i + 1); int len = Math.max(len1, len2); if (len > end - start) { start = i - (len - 1) / 2; end = i + len / 2; } } return s.substring(start, end + 1); } private static int expand(String s, int L, int R) { while (L >= 0 && R < s.length() && s.charAt(L) == s.charAt(R)) { L--; R++; } return R - L - 1; }
```

**Review Check:** Prompting specifically for expand-around-center reduces memory overhead and passes strict online judge constraints.

## Module 2: Arrays, Searching & Math Logic

### Q7. Find Missing Number in Continuous Sequence [1 to N]

MATH / BITWISE XOR

**Optimal Student Prompt:** "Write a Java method to detect the single missing integer from an array containing N-1 distinct integers in the range 1 to N. Use bitwise XOR logic to prevent 32-bit integer overflow issues with large sum formulas."

```
public static int findMissingNumber(int[] nums, int n) { int xor = 0; for (int i = 1; i <= n; i++) xor ^= i; for (int num : nums) xor ^= num; return xor; // Remaining XOR value is exactly the missing number }
```

**Review Check:** Point out to students that formula `n*(n+1)/2` causes integer overflow when n > 65,536; XOR is bulletproof.

### Q8. Two Sum: Pair Target Sum Detection

HASH TABLE

**Optimal Student Prompt:** "Implement a C++ solution returning indices of two elements summing up to target in an unsorted array. Achieve O(n) time using `unordered_map`. Handle edge case where no valid pair exists by returning empty vector."

```
vector<int> twoSum(vector<int>& nums, int target) { unordered_map<int, int> map; for (int i = 0; i < nums.size(); i++) { int complement = target - nums[i]; if (map.find(complement) != map.end()) return {map[complement], i}; map[nums[i]] = i; } return {}; }
```

**Review Check:** AI often uses duplicate elements twice if map is populated in advance; single-pass map lookup prevents reusing index.

### Q9. Maximum Subarray Sum (Kadane's Algorithm)

OPTIMIZATION

**Optimal Student Prompt:** "Write a Java solution using Kadane's algorithm to calculate the maximum subarray sum in an array containing both positive and negative values. Ensure all-negative arrays return the single highest negative value."

```
public static int maxSubArray(int[] nums) { int maxSoFar = nums[0], currMax = nums[0]; for (int i = 1; i < nums.length; i++) { currMax = Math.max(nums[i], currMax + nums[i]); maxSoFar = Math.max(maxSoFar, currMax); } return maxSoFar; }
```

**Review Check:** Initializing `maxSoFar = 0` causes failure on all-negative inputs like [-5, -2, -8]; must initialize to `nums[0]`.

## Module 2 Continued: Array Shifts, Inversions & Matrix Operations

### Q10. Rotate Array Left/Right by K Positions In-Place

ARRAY ROTATION

> **Optimal Student Prompt:** "Write a Java program to rotate an integer array of size N right by K steps in-place. Use the 3-step reversal algorithm to guarantee O(n) runtime and O(1) extra space. Ensure K is normalized using modulo `k = k % n`."

```
public static void rotate(int[] nums, int k) { k = k % nums.length; reverse(nums, 0, nums.length - 1); reverse(nums, 0, k - 1); reverse(nums, k, nums.length - 1); } private static void reverse(int[] a, int l, int r) { while (l < r) { int temp = a[l]; a[l++] = a[r]; a[r--] = temp; } }
```

**Review Check:** Ensure normalization `k %= n` prevents IndexOutOfBoundsException when k is substantially larger than array length.

### Q11. Separate Even and Odd Integers In-Place

PARTITIONING

> **Optimal Student Prompt:** "Write a C++ function that segregates even numbers to the front and odd numbers to the back of an array in-place without creating secondary arrays. Use a two-pointer Dutch National Flag style partition."

```
void segregateEvenOdd(vector<int> &arr) { int left = 0, right = arr.size() - 1; while (left < right) { while (arr[left] % 2 == 0 && left < right) left++; while (arr[right] % 2 != 0 && left < right) right--; if (left < right) { swap(arr[left], arr[right]); left++; right--; } } }
```

**Review Check:** Check inner while-loop boundary bounds (`left < right`) to avoid runaway pointers on arrays containing only even or odd items.

### Q12. Vehicle Wheel Counting Problem (Capgemini Arithmetic Pattern)

LINEAR EQUATIONS

> **Optimal Student Prompt:** "Write a Java solution for Capgemini's vehicle wheel question: given V vehicles and W wheels, compute count of 2-wheelers and 4-wheelers. Return -1 or invalid if W is odd, W < 2*V, or W > 4*V. Time O(1)."

```
public static void solveVehicles(int V, int W) { if (W % 2 != 0 || W < 2 * V || W > 4 * V) { System.out.println("INVALID INPUT"); return; } int fourWheelers = (W - (2 * V)) / 2; int twoWheelers = V - fourWheelers; System.out.println("2-Wheelers: " + twoWheelers + ", 4-Wheelers: " + fourWheelers); }
```

**Review Check:** Direct student to audit equation algebra: 2x + 4y = W, x + y = V => y = (W - 2V)/2.

## Module 3: Sorting, Two Pointers & Hashing

### Q13. Merge Two Sorted Arrays Without Extra Space

GAP METHOD / SHELL

> **Optimal Student Prompt:** "Write a C++ program to merge two sorted arrays `arr1` of size m and `arr2` of size n in-place with O(1) extra space. Use the Gap method (Shell sort concept) to achieve O((n+m) log(n+m)) time complexity."

```
void mergeInPlace(int arr1[], int arr2[], int n, int m) { int gap = (n + m + 1) / 2; while (gap > 0) { int i = 0, j = gap; while (j < (n + m)) { int *a = (i < n) ? &arr1[i] : &arr2[i - n]; int *b = (j < n) ? &arr1[j] : &arr2[j - n]; if (*a > *b) swap(*a, *b); i++; j++; } gap = (gap <= 1) ? 0 : (gap + 1) / 2; } }
```

**Review Check:** AI often defaults to copying both arrays into a new vector. Insist on the Gap technique for zero-memory compliance.

### Q14. Find Majority Element (Moore's Voting Algorithm)

VOTING ALGORITHM

> **Optimal Student Prompt:** "Write a Java solution to identify the majority element appearing more than n/2 times using Boyer-Moore Voting Algorithm in O(n) time and O(1) space. Include a verification pass if existence is not guaranteed."

```
public static int majorityElement(int[] nums) { int candidate = 0, count = 0; for (int num : nums) { if (count == 0) candidate = num; count += (num == candidate) ? 1 : -1; } count = 0; for (int num : nums) if (num == candidate) count++; return (count > nums.length / 2) ? candidate : -1; }
```

**Review Check:** Ensure the second verification loop is present; without it, non-majority arrays produce incorrect candidates.

### Q15. Intersection of Two Sorted Arrays Without Duplicates

TWO POINTERS

> **Optimal Student Prompt:** "Write a Java function taking two sorted arrays and returning their intersection containing unique common values only. Use standard two pointers in O(n+m) time without utilizing Set collections."

```
public static List<Integer> intersection(int[] a, int[] b) { List<Integer> res = new ArrayList<>(); int i = 0, j = 0; while (i < a.length && j < b.length) { if (i > 0 && a[i] == a[i - 1]) { i++; continue; } if (a[i] < b[j]) i++; else if (b[j] < a[i]) j++; else { res.add(a[i]); i++; j++; } } return res; }
```

**Review Check:** Duplicate handling: check `i > 0 && a[i] == a[i-1]` skips redundant matches seamlessly.

## Module 4: Linked Lists, Stacks & Matrix Coordinates

### Q16. Reverse a Singly Linked List Iteratively

POINTERS

> **Optimal Student Prompt:** "Write a Java method `ListNode reverseList(ListNode head)` to reverse a singly linked list iteratively in O(n) time and O(1) space. Explicitly define ListNode and handle null/single-node edge cases safely."

```
public ListNode reverseList(ListNode head) { ListNode prev = null, curr = head; while (curr != null) { ListNode nextTemp = curr.next; curr.next = prev; prev = curr; curr = nextTemp; } return prev; }
```

**Review Check:** Confirm pointers update in strict order: store next, flip next, advance prev, advance curr.

### Q17. Valid Parentheses Matching with Multiple Bracket Types

STACK

> **Optimal Student Prompt:** "Write a C++ function `bool isValid(string s)` checking balanced parentheses for '()', '[]', and '{}'. Use `std::stack`. Reject immediately if string length is odd. Handle empty stack pops gracefully."

```
bool isValid(string s) { if (s.length() % 2 != 0) return false; stack<char> st; for (char c : s) { if (c == '(' || c == '{' || c == '[') st.push(c); else { if (st.empty()) return false; char top = st.top(); st.pop(); if ((c == ')' && top != '(') || (c == '}' && top != '{') || (c == ']' && top != '[')) return false; } } return st.empty(); }
```

**Review Check:** Empty stack underflow check (`st.empty()`) before popping is essential; AI code often crashes on input like `")("`.

### Q18. Matrix Transpose & 90-Degree Clockwise Rotation In-Place

2D MATRIX

> **Optimal Student Prompt:** "Provide a Java method to rotate an N x N 2D matrix clockwise by 90 degrees in-place without allocating a new matrix. Guide: First transpose across diagonal, then reverse each row. Target O(n^2) time, O(1) space."

```
public static void rotateMatrix(int[][] matrix) { int n = matrix.length; for (int i = 0; i < n; i++) { for (int j = i + 1; j < n; j++) { int t = matrix[i][j]; matrix[i][j] = matrix[j][i]; matrix[j][i] = t; } } for (int i = 0; i < n; i++) { for (int j = 0; j < n / 2; j++) { int t = matrix[i][j]; matrix[i][j] = matrix[i][n - 1 - j]; matrix[i][n - 1 - j] = t; } } }
```

**Review Check:** Ensure transpose inner loop starts at `j = i + 1` instead of `0` to prevent double-swapping back to original.

## Module 5: Dynamic Programming & Binary Search Essentials

### Q19. Coin Change: Minimum Coins Required

DP / UNBOUNDED KNAPSACK

**Optimal Student Prompt:** "Write a Java solution to find minimum coins required to make amount A given coin denominations array. Use 1D bottom-up DP table. Return -1 if amount cannot be formed. Initialize table with amount+1."

```
public static int coinChange(int[] coins, int amount) { int[] dp = new int[amount + 1]; Arrays.fill(dp, amount + 1); dp[0] = 0; for (int i = 1; i <= amount; i++) { for (int coin : coins) if (i - coin >= 0) dp[i] = Math.min(dp[i], 1 + dp[i - coin]); } return dp[amount] > amount ? -1 : dp[amount]; }
```

**Review Check:** Using `Integer.MAX_VALUE` causes integer overflow when adding `1 + dp[...]`; use `amount + 1` as infinity sentinel.

### Q20. Binary Search on Rotated Sorted Array

MODIFIED BINARY SEARCH

**Optimal Student Prompt:** "Write a C++ function `int searchRotated(vector<int>& nums, int target)` that finds target in a rotated sorted array in O(log n) time. Determine which half is sorted, then narrow boundaries."

```
int searchRotated(vector<int>& a, int target) { int l = 0, r = a.size() - 1; while (l <= r) { int mid = l + (r - l) / 2; if (a[mid] == target) return mid; if (a[l] <= a[mid]) { if (target >= a[l] && target < a[mid]) r = mid - 1; else l = mid + 1; } else { if (target > a[mid] && target <= a[r]) l = mid + 1; else r = mid - 1; } } return -1; }
```

**Review Check:** Confirm midpoint formula `l + (r - l) / 2` is used rather than `(l + r) / 2` to prevent standard arithmetic overflow.

### Q21. Climbing Stairs (Fibonacci DP Variant)

STATE OPTIMIZATION

**Optimal Student Prompt:** "Write a Java method `climbStairs(int n)` returning distinct ways to climb N stairs taking 1 or 2 steps. Optimize to O(n) time and O(1) space using two rolling integer variables instead of array."

```
public static int climbStairs(int n) { if (n <= 2) return n; int a = 1, b = 2; for (int i = 3; i <= n; i++) { int c = a + b; a = b; b = c; } return b; }
```

**Review Check:** Base cases: N=1 returns 1, N=2 returns 2. Space optimization demonstrates superior code efficiency.

### Q22. Detect Cycle in a Singly Linked List

FLOYD'S TORTOISE & HARE

**Optimal Student Prompt:** "Write a Java solution detecting if a cycle exists in a linked list using Floyd's Tortoise and Hare algorithm. Fast pointer moves 2 steps, slow moves 1 step. Space O(1), time O(n)."

```
public boolean hasCycle(ListNode head) { ListNode slow = head, fast = head; while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; if (slow == fast) return true; } return false; }
```

**Review Check:** Check while-condition `fast != null && fast.next != null` prevents NullPointerExceptions.

## Module 6: Capgemini Special Numerical Problems & Trainer Cheat Sheet

### Q23. Special Number / Armstrong Number Verification

MATH / NUMBER THEORY

**Optimal Student Prompt:** "Write a Java program to check whether a given positive integer N is an Armstrong number (sum of digits raised to the power of total digits equals N). Avoid Math.pow double cast errors."

```
public static boolean isArmstrong(int n) { int original = n, temp = n, digits = 0, sum = 0; while (temp > 0) { digits++; temp /= 10; } temp = n; while (temp > 0) { int r = temp % 10; int p = 1; for (int i = 0; i < digits; i++) p *= r; sum += p; temp /= 10; } return sum == original; }
```

**Review Check:** Integer power loop prevents precision loss caused by IEEE-754 floating point arithmetic in `(int)Math.pow()`.

### Q24. Sum of Prime Numbers in Range [L, R] (Sieve of Eratosthenes)

PRIMES / MATH

**Optimal Student Prompt:** "Write a C++ program computing the sum of all primes between L and R inclusive where R <= 10^6. Use the Sieve of Eratosthenes to precompute primes in O(R log log R) instead of trial division per element."

```
long long primeSumRange(int L, int R) { vector<bool> isPrime(R + 1, true); isPrime[0] = isPrime[1] = false; for (int p = 2; p * p <= R; p++) { if (isPrime[p]) { for (int i = p * p; i <= R; i += p) isPrime[i] = false; } } long long sum = 0; for (int i = max(2, L); i <= R; i++) if (isPrime[i]) sum += i; return sum; }
```

**Review Check:** Using `long long` for sum avoids 32-bit sign overflow on high ranges; inner sieve starts at `p * p`.

### Q25. Evaluate Algebraic Expression: (a+b)^3 Expansion

CAPGEMINI FREQUENT

**Optimal Student Prompt:** "Write a Java solution for Capgemini's algebraic expression: given inputs a and b, compute result of a^3 + 3*a^2*b + 3*a*b^2 + b^3 using algebraic simplification (a+b)^3 to guarantee O(1) operations."

```
public static long evaluateExpression(long a, long b) { long sum = a + b; return sum * sum * sum; // Simplified (a+b)^3 prevents multiple redundant powers }
```

**Review Check:** Teaching shortcut: recognizing algebraic identities simplifies 8 separate operations down to a single cubic multiplication.

**Classroom Instruction Checklist for AI-Assisted Assessment:**

- **Enforce the 4-part Prompt Template:** Goal Statement + Language Selection + Space/Time Constraints + Edge Cases.
- **Warn against vague prompts:** Typing *"Write code for question 1"* yields inefficient O(n²) implementations that timeout on hidden test cases.
- **Code Review is 40% of the grade:** Students must inspect AI outputs for unhandled null pointers, array index overflow, and off-by-one errors before running test suites.

## Module 7: Foundational Level MCQs (Questions 1 to 4)

**Classroom Mode (One-by-One Presentation):** Pose each question to the batch first. Click the disclosure box to reveal the exact matching answer and coaching point. FOUNDATIONAL TIER

**Question 1: What is the worst-case time complexity of searching for an element in an unsorted array of size N?**

**A)** O(1)

**B)** O(log N)

**C)** O(N)

**D)** $O(N^2)$

**Question 2: In AI-assisted coding assessments, what happens if your prompt does NOT mention boundary cases like empty strings or single-element inputs?**

**A)** The AI compiler halts and throws a syntax error

**B)** The AI typically generates logic for the happy path only, failing hidden edge-case tests with NullPointerException or IndexOutOfBounds

**C)** The testing platform automatically sanitizes and injects dummy data

**D)** The AI assistant refuses to generate any code

**Question 3: Which data structure operates strictly on a Last-In, First-Out (LIFO) order of elements?**

**A)** Queue

**B)** Array

**C)** Stack

**D)** Priority Queue

**Question 4: Given a string of length N, what is the exact formula for the total number of non-empty contiguous substrings?**

**A)** N

**B)** $2^N$

**C)** N * (N + 1) / 2

**D)** $N^2$

## Module 7 Continued: Foundational & Intermediate MCQs (Questions 5 to 8)

**Question 5: Which operator in C++ and Java computes the integer remainder after a division operation?**

**A)** /

**B)** %

**C)** //

**D)** &

**Question 6: Which student prompt will produce the most optimal, bug-free in-place array reversal?**

**A)** "Reverse this array quickly in Java"

**B)** "Write code for reversing numbers"

**C)** "Write an in-place two-pointer Java function to reverse an integer array in O(N) time and O(1) auxiliary space, handling null and empty inputs safely"

**D)** "Give me solution for Capgemini test question 1"

**Question 7: When asking an AI assistant to implement Kadane's algorithm, what edge case does it frequently mishandle?**

**A)** An array containing all positive integers

**B)** An array consisting exclusively of negative integers (returning 0 instead of the single largest negative number)

**C)** An array with an odd number of elements

**D)** An array containing both zeros and positive numbers

**Question 8: An AI assistant generated: `int mid = (low + high) / 2;` for a binary search. What flaw must the candidate identify?**

**A)** Syntax error in modern C++ and Java compilers

**B)** Potential 32-bit signed integer overflow when `low + high` exceeds $2^{31} - 1$

**C)** Mid will always evaluate to zero

**D)** Mid will be converted to a floating point number

## Module 8: Advanced Assessment MCQs (Questions 9 to 12)

**Question 9: To find a missing number in a sequence [1..N] without risk of integer overflow when N = $10^6$, which strategy must be prompted?**

**A)** "Calculate sum using N * (N + 1) / 2 in a standard 32-bit signed integer"

**B)** "Sort the array in O(N log N) time and scan indices sequentially"

**C)** "Compute the bitwise XOR of all numbers from 1 to N and XOR with all array elements"

**D)** "Use a nested loop comparing each element against all numbers from 1 to N"

**Question 10: In Floyd's Cycle Detection generated by an AI assistant, which while-loop condition guarantees no NullPointerException?**

**A)** `while (slow != null)`

**B)** `while (fast != null && fast.next != null)`

**C)** `while (fast.next != null)`

**D)** `while (slow == fast)`

**Question 11: In the Coin Change Dynamic Programming problem, why is initializing the DP table with `Integer.MAX_VALUE` dangerous?**

**A)** Java arrays cannot store values equal to Integer.MAX_VALUE

**B)** Adding 1 in `1 + dp[i - coin]` causes 32-bit signed integer wraparound to a large negative number

**C)** It triggers an immediate JVM OutOfMemoryError

**D)** It prevents the compiler from optimizing the loop

**Question 12: What is the optimal time and auxiliary space complexity achieved by the Dutch National Flag algorithm when segregating 0s, 1s, and 2s?**

**A)** O(N log N) time and O(1) auxiliary space

**B)** O(N) time and O(N) auxiliary space

**C)** O(N) time and O(1) auxiliary space

**D)** $O(N^2)$ time and O(1) auxiliary space

## Trainer Master Answer Key & Classroom Delivery Plan

Keep this reference page on your trainer desk or tablet during classroom sessions. Students will only see the questions and options until you call on them to explain their reasoning.

| Q# | Topic & Core Concept | Correct | Teaching / Prompting Takeaway to Explain to Batch |
|---|---|---|---|
| **Q1** | Unsorted Array Search | **C** | Unsorted array search requires scanning each element: O(N). Binary Search O(log N) requires sorted data. |
| **Q2** | AI Prompt Edge-Cases | **B** | AI defaults to happy path logic. Emphasize appending "Handle null/empty inputs" to every prompt. |
| **Q3** | LIFO Data Structure | **C** | Stack enforces LIFO. Key for balanced brackets (Q17) and recursive call stack simulation. |
| **Q4** | Contiguous Substrings | **C** | Contiguous substrings = N*(N+1)/2. Non-contiguous subsequences = $2^N$ - 1. |
| **Q5** | Remainder Operator | **B** | Modulo operator `%` computes division remainder. Essential for circular arrays (k % n). |
| **Q6** | Prompt Formulation | **C** | Effective prompts specify pattern (two-pointer), language, complexity, and null/empty handling. |
| **Q7** | Kadane All-Negative Array | **B** | Initializing max to 0 returns 0 on negative arrays. Always initialize with nums[0]. |
| **Q8** | Binary Search Midpoint Overflow | **B** | (low + high) overflows 32-bit signed integer. Replace with low + (high - low) / 2. |
| **Q9** | Missing Number XOR Strategy | **C** | Sum formula overflows 32-bit int on large N. XOR property (a ^ a = 0) cannot overflow. |
| **Q10** | Floyd Cycle Pointer Guard | **B** | Must verify both fast and fast.next are non-null before advancing fast pointer 2 steps. |
| **Q11** | DP Table MAX_VALUE Overflow | **B** | MAX_VALUE + 1 wraps to -2,147,483,648, corrupting Math.min(). Initialize with amount + 1. |
| **Q12** | Dutch National Flag Complexity | **C** | Dutch National Flag sorts 3 partitions in single linear pass O(N) with O(1) extra space. |

**Classroom Interactive Drill Protocol:**

1. **Step 1 (Display):** Project Page 9, 10, or 11 on the screen or distribute handouts without Page 12.
2. **Step 2 (Poll):** Give students 45 seconds to choose an option individually.
3. **Step 3 (Prompt Challenge):** Ask 1 student: *"If you were prompting an AI for this question, what exact words would you include to prevent bugs?"*
4. **Step 4 (Reveal & Explain):** Click the interactive reveal summary (or reference the table above) to debrief the solution.